# Sequence-Based Emotion Recognition from Natural Language

**Dhruv Patel**
University of Toronto Mississauga
Mississauga, ON L5L 1C6
dhruvvv.patel@mail.utoronto.ca

**Kirtan Patel**
University of Toronto Mississauga
Mississauga, ON L5L 1C6
kirtan.patel@mail.utoronto.ca

**Ethan Tang**
University of Toronto Mississauga
Mississauga, ON L5L 1C6
eth.tang@mail.utoronto.ca

**Mingzhi Dong**
University of Toronto Mississauga
Mississauga, ON L5L 1C6
mingzhi.dong@mail.utoronto.ca

## Abstract

Understanding the emotions expressed in text is important in applications such as mental health assessment, customer feedback analysis, and human computer interaction. This project uses the publicly available emotions dataset for NLP (Natural Language Processing) on Kaggle, to explore the problem of detecting emotions from short and user generated text. The dataset provides labeled sentence instances that contain various emotion categories, such as joy, anger, sadness, fear, and love. We compare two neural network architectures: Recurrent Neural Networks (RNNs) and Transformer based models to examine how sequence based and attention based models capture emotional cues in short, informal sentences. These models are all trained and evaluated on this dataset, and our primary goal is to gain insights into the trade-offs between model complexity, interpretability, and classification performance in real-world text sentiment recognition tasks.

## 1 Introduction

Understanding the emotions expressed in text is an important topic in Natural Language Processing, where applications span areas such as mental health support, customer feedback analysis, and human computer interaction. This sentiment detection goes beyond traditional sentiment analysis, which simply tells and detects if text with positive or negative emotion; This project focuses on identifying more specific emotional states, such as joy, sadness, anger, fear, and love, so that it can provide a deeper understanding of human emotions in language. Building models capable of accurately

Preprint. Under review.

detecting emotions helps to develop more empathetic AI systems and improve tools for analyzing people's emotional expressions online.

Despite having a wide range of applications, sentiment detection is still a challenging task. People often express emotions indirectly through the subtle choice of words. In addition, short sentences generated by users can also contain spelling errors, informal language, and ambiguous emotional expressions, making it difficult for models to understand their actual emotion. Therefore, sentiment detection provides a valuable method for testing the ability of modern NLP models to handle complex, real-world language.

In this project, we will use a publicly available dataset on Kaggle which contains short sentences and labeled with different emotions. We will compare two neural network architectures: RNNs and Transformer based models to investigate their emotion detect abilities, and how they each capture sentiment cues in text. By training and evaluating both models on the same dataset, we want to understand their relative strengths and weaknesses in terms of accuracy, interpretability, and complexity. Our findings can provide insight into how neural network models represent sentiment, and may help guide the construction of better sentiment aware natural language processing systems in the future.

## 2 Background & related works

A crucial area of Natural Language Processing (NLP) that goes beyond sentiment analysis is text-based emotion recognition. Emotion detection seeks to categorize text into specific psychological categories like joy, anger, fear, sadness, and disgust, whereas sentiment analysis usually divides text into broad polarities (positive, negative, or neutral) (Bharti et al., 2022). The multi-class nature of the problem and the lack of non-verbal indicators (such as tonal tension or facial expressions) present in voice or video data make this task intrinsically more difficult.

Keyword and lexicon-based approaches, which depended on pre-established dictionaries of emotive terms, were the first to be used in this arena. However, because these approaches are unable to capture semantic context, they frequently fall short when confronted with sarcasm, negation, or confusing phrasing (Bharti et al., 2022). As a result, the discipline shifted to traditional Machine Learning (ML) techniques like Naive Bayes and Support Vector Machines (SVM). Although these provided advancements, they were nevertheless constrained by their dependence on manual feature engineering and their incapacity to accurately represent intricate language connections.

Deep Learning (DL) is a key component of the present state-of-the-art for automating feature extraction and capturing sequential context. This move toward DL is not exclusive to text; comparable structures have shown great success in related fields. For example, (Grekow, 2021) demonstrated that Long Short-Term Memory (LSTM) networks significantly outperformed simpler machine learning baselines when predicting emotion from sequential audio features by using Recurrent Neural Networks (RNNs) to music emotion detection.

(Bharti et al., 2022) also used Bidirectional Gated Recurrent Units (Bi-GRUs) for text analysis because of RNNs' cross-domain success, which validates their usefulness for processing sequential data. In their research, they showed that integrating CNNs with Bi-GRUs produced a notable improvement in performance over conventional ML baselines, reaching about 80% accuracy on common datasets.

However, while RNN-based architectures like GRUs and LSTMs are effective for sequences, they can struggle with training efficiency and long-range dependencies compared to modern Transformer architectures. By developing and contrasting Transformer-based models with Recurrent Neural Network (RNN) baselines, our effort seeks to expand on these deep learning foundations. We examine whether the attention mechanisms built into Transformers offer a noticeable advantage in identifying emotional nuance when compared to the sequential processing of conventional RNNs, using a large-scale dataset of English messages labeled with six fundamental emotions (anger, fear, joy, love, sadness, and surprise).

## 3 Data

The dataset used in this project is the Emotion Dataset for NLP by Praveen Govindaraj and publicly available on Kaggle [1]. It contains short informal text messages paired with one of six emotion labels:

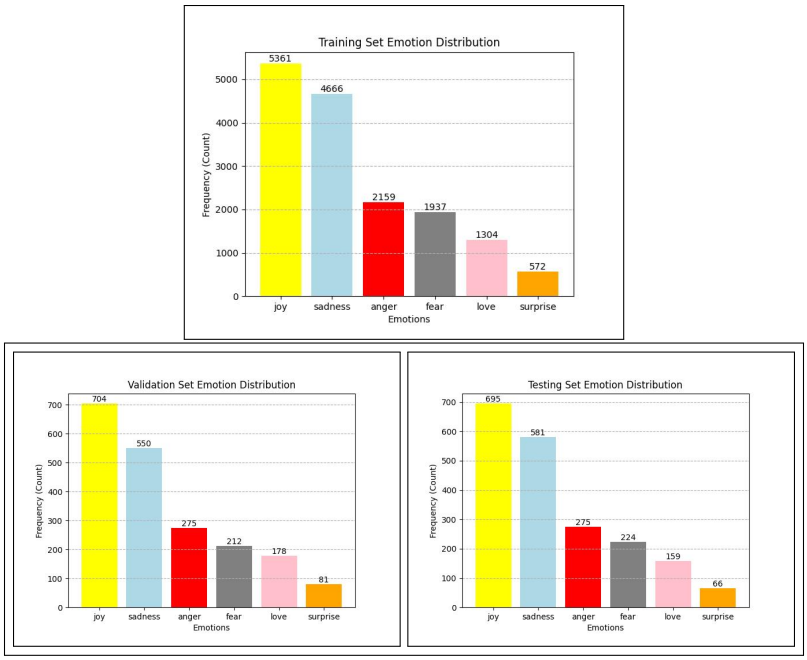


Figure 1: Emotion label frequency across the training, validation, and testing splits. The distributions are consistent across splits, showing that no single label dominates the dataset.

anger, fear, joy, love, sadness, and surprise. This dataset is already split into training, validation, and test sets (~16k, 2k, and 2k sample entries respectively), upon acquisition. To determine whether this data is split correctly, we examined the frequency of each label by categorizing the text of each emotion.

### 3.1 Distribution of emotion labels

This step helps us visualize the balance between each class within the split sets and understand whether the patterns and distribution of each label is consistent in each split. The distribution is not perfectly uniform as joy and sadness appear most frequently, while love and surprise do not, but we can conclude that no single label overwhelms the dataset in its entirety. This means that the model must pick up and learn from meaningful linguistic patterns rather than simply predicting the most common label. Additionally, since the dataset is complied with short and informal text sequences, to capture the emotion the model must be context dependent when making predictions. This class distribution is shown in Figure 1.

### 3.2 Data cleaning and preprocessing

Before any data cleaning and processing, we converted each file (train.txt, val.txt, test.txt) into CSV format with two columns: text and emotion. This was necessary to make the data easier to manipulate in python and indexing for preprocessing.

As part of the cleaning process, we convert the text to lowercase, removed leading and tailing white spaces and duplicate entries, which prevent the model from treating equivalent strings differently (happy vs. Happy) and ensure no sample is overrepresented. Other than these basic normalization steps, we preformed three additional preprocessing steps.

**Removing punctuations and numbers** Since punctuation marks and numeric character do not contribute meaningful emotional information, they are removed from the data. Additionally, some entries use punctuation, and some do not, so to avoid the model treating this pattern as meaningful or picking up on inconsistent structure, we removed all punctuation. For example, “i’m feeling so sad today!!” becomes “im feeling so sad today”.

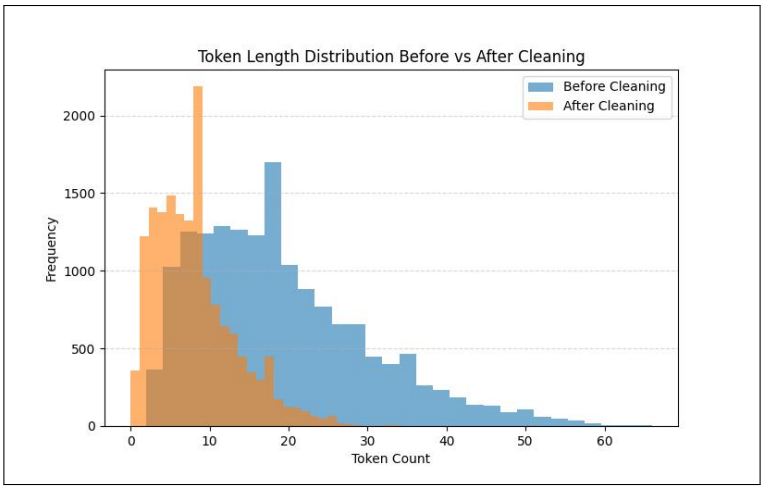


Figure 2: Token length distribution before and after cleaning, showing shorter and more focused text sequences.

**Stopword filtering with negation prevention** Stopword are common words that often hold little to no standalone meaning which can add noise. However, in this case, negation words like not, don't, and won't are important because they can change the emotional meaning of the text. To prevent this, we modified the list of standard English stopword to remove negation words. After this step, the text becomes more compact and focused on word that contribute meaningful emotional information. For example, "im feeling so sad today" becomes "feeling sad today".

**Tokenization** To make the dataset suitable for processing, the text was tokenized into individual words, to create a sequence that could then be fed to models to process word order. For example, "feeling sad today" becomes ["feeling", "sad", "today"].

### 3.3 Effect of cleaning on token length

After cleaning, the average sentence becomes shorter and contains fewer filler words as shown in Figure 2. This reduces noise and improves the model's concentration on emotion keywords. Resulting in a sequence that has a clear and meaningful signal without any unnecessary padding or irrelevant words.

## 4 Model architecture

We will use both a Long Short Term Memory Recurrent Neural Network (LSTM RNN) and an encoder-only Transformer to perform sentiment analysis on short sentences.

### 4.1 LSTM RNN architecture

We chose an LSTM RNN model due to its capability of modelling long-term dependencies between words in a sentence. This is particularly important for sentiment analysis, where understanding the contextual relationships between words is pivotal for accurately classifying the sentiment of a sentence. The LSTM RNN model architecture is illustrated on the left of Figure 3.

Our LSTM RNN begins with an input embedding layer followed by a bidirectional LSTM layer. Next, the hidden states undergo max and mean pooling before being classified by a linear layer to produce logits for sentiment classification.

**Input Embedding:** We use an embedding layer to convert the input tokens to learned embedding vectors of dimension 128. The embedding layer is a simple lookup table that maps each token in our vocabulary to its respective embedding vector. The embedding weights are initialized randomly and optimized during training.

**Bidirectional LSTM:** We use a single bidirectional LSTM layer with 128 hidden units to process the embedding vectors in both the forward and backward directions to capture contextual information from both preceding and subsequent tokens.

**Pooling:** We apply both max and mean pooling over the 256-dimensional sequence of hidden states (128 from each direction) produced by the bidirectional LSTM layer and concatenate the results. Max pooling captures the most prominent features in a sequence, while mean pooling captures the general features in a sequence, which combined provide a more complete picture of the sequence.

**Linear Layer:** A single fully connected layer that maps the 512-dimensional pooled features (256 from each of max and mean pooling) to 6 emotion classes (anger, fear, joy, love, sadness, surprise).

#### 4.1.1 LSTM RNN Training Configuration

We trained our model with cross-entropy loss and the Adam optimizer for ten epochs. We used the following hyperparameters: embedding size of 128, hidden size of 128, dropout rate of 0.3, learning rate of 0.01, and batch size of 64. Additionally, we had a vocabulary size of 7,232 tokens.

### 4.2 Transformer Architecture

We chose an encoder-only Transformer because of its ability to capture relationships between all words in a sentence simultaneously using self-attention, which allows the model to understand contextual relationships between words better than if they were processed sequentially as with RNNs. The encoder-only Transformer model architecture is illustrated on the right of Figure 3.

Our encoder-only Transformer begins with an input embedding layer combined with positional encodings to represent both the meaning of each token and their order in the sentence. The embedding vectors are then passed through two encoder layers, each of which applies multi-head self-attention and a feed-forward network to obtain contextualized embeddings for each token. We then apply mean pooling across these contextualized embeddings to obtain a single vector that represents the entire sentence. Finally, this sentence-level vector is passed through a linear layer to generate logits for sentiment classification.

**Input Embedding + Positional Encoding:** Similar to our LSTM RNN implementation, we convert each input token into a 128-dimensional learned embedding. Furthermore, we scale each embedding by $\sqrt{d_{model}} = \sqrt{128}$ before adding sinusoidal positional encodings to balance the relative magnitudes while encoding positional information.

**Encoder:** Our encoder consists of a stack of $N = 2$ layers. Each encoder layer has two sub-layers: a multi-head self-attention layer and a position-wise feed-forward network. The multi-head self-attention layer is implemented with four attention heads, allowing the model to capture multiple types of contextual relationships between tokens. The position-wise feed-forward network is used to process each token representation independently to refine the contextualized embeddings for each token.

**Pooling:** We apply mean pooling over the 128-dimensional sequence of contextualized embeddings produced by the encoder layers. We use mean pooling to capture the general features across the sequence.

**Linear Layer:** A single fully connected layer that maps the 128-dimensional pooled features to 6 emotion classes (anger, fear, joy, love, sadness, surprise).

#### 4.2.1 Transformer Training Configuration

We trained our model with cross-entropy loss and the Adam optimizer for ten epochs using the following hyperparameters: model dimension ($d_{model}$) of 128, 4 attention heads, 2 encoder layers, dropout rate of 0.1, learning rate of 2e-4, feed-forward network dimension of 256, and batch size of 64. Additionally. we had a vocabulary size of 7,121 tokens and we truncated each input sequence to be at most 40 tokens long.

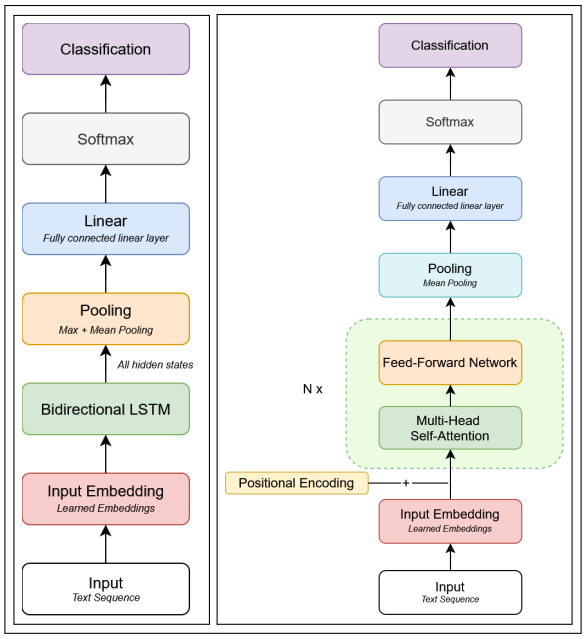


Figure 3: LSTM RNN Architecture & Encoder-only Transformer Architecture

### 4.3 Baseline RNN Model

We used a simple vanilla RNN model as a baseline for comparison. This baseline model consists of an embedding layer (100 dimensions), a vanilla RNN layer (64 hidden units), max and mean pooling over the hidden states, and a linear layer.

#### 4.3.1 Baseline RNN Training Configuration

We trained this model with cross-entropy loss and the Adam optimizer for fifteen epochs using the following hyperparameters: embedding size of 100, hidden size of 64, learning rate of 0.01, and batch size of 64.

## 5 Results

To evaluate model performance on how well our it classifies emotions, we examine their precision, recall, and F1-score for all six classes of emotion, as well as their weighted averages to account for class imbalance in our data sets, where the labels joy and sadness are more common than the rest. These metrics provide a better idea of performance and per-class behaviour than accuracy alone, especially when considering that there are relatively few observations in classes like love and surprise. All results used below are taken from our test data set of 2000 observations.

As a baseline, we use a Vanilla RNN, which represents the simplest recurrent architecture capable of learning from sequential dependencies. We found this model to be a reasonable baseline because of these two following reasons. Firstly, it provides a lower bound for a sequence-based learning task. Secondly, it provides a point of comparison to identify how much of a performance improvement can be gained due to additional architectural components, namely gating in LSTMs and attention in Transformers.

### 5.1 Overall performance comparison

The performance of the LSTM outperforms all other weighted metrics, with a slight lead of 2-3 F1 values over the RNN baseline. The performance of the Transformer also seems quite good, but it falls a little behind the performance of the LSTM for this problem. Given the dataset's short and informal

Table 1: Summarizes the weighted metrics for all models

| Model | Accuracy | Weighted Precision | Weighted Recall | Weighted F1 |
|---|---|---|---|---|
| Vanilla RNN | 0.87 | 0.8737 | 0.8715 | 0.8718 |
| LSTM | 0.90 | 0.8992 | 0.8995 | 0.8987 |
| Transformer | 0.8685 | 0.8694 | 0.8685 | 0.8686 |

sentences, it seems that the use of gates in LSTMs has a better impact than the use of attention in Transformers, which typically excels with longer sentences.

### 5.2 Per-class analysis

**Vanilla RNN model** provides a solid baseline, performing well on high observation classes like joy (F1=0.90) and sadness (F1=0.91) but struggling with low observation classes like love and surprise, where F1 reduces to 0.73 and 0.69, respectively. This pattern suggests that despite being able to capture general sentiment knowledge, it does not provide sufficient representational capacity for capturing subtle expressions of certain emotions.

**LSTM RNN model** clearly outperforms the baseline in almost all classes. The model performs better in terms of F1 values in anger, fear, joy, and sadness, with a major focus on significantly improving the value of love, which reached 0.77, a significant improvement considering how subtle this class is. The structure of the model, which has a gating mechanism, seems to benefit the retention of contextual data in this small set of data, as it assists in classifying subtle expressions of emotion.

**Transformer model** performs well on prominent classes with F1-values of 0.88 for anger, 0.89 for joy, and 0.91 for sadness but has a higher variance for all other labels. Its performance decreases for love with an F1 of 0.75 and surprise with an F1 of 0.67, suggesting that it struggles with emotion classes with fewer observations as it becomes difficult for the attention model to function effectively. Although it nearly reaches the performance of LSTM on some dominant classes, it lacks generalization capabilities for all six types of emotions.

### 5.3 Qualitative interpretation

Throughout all three models, the misclassifications often occur in sentences where emotions are implicit or mixed, for example, sentences that express frustration indirectly or combining multiple feelings at once. In sentences that are composed of a few words with no prominent affective keywords, there are usually uncertainties, especially in the baseline RNN. The LSTM model performs better in these situations, particularly in instances where it needs to identify subtle changes in sentences with emotionally nuanced words. On the other hand, for models like the Transformer, there might rely overly on key isolated words rather than the message, which can be misleading, especially when analyzing words that are linked to contexts in sentences with feelings of love and surprise.

## 6 Discussion

The differences in performance between our models make it clear that architectural complexity greatly depends on the structure and size of a given set of data. While the LSTM clearly has exceptional and consistent performance, this seems to be due to the gating mechanism effectively capturing a situation where a nuance of a text, rather than a long-term dependency, yields a corresponding emotional classification. Although a good baseline model, the Vanilla RNN has issues with minor emotions, similar to how it has limited capabilities for maintaining detailed information, particularly deteriorating into a drift towards majority classes. While a contender for high-support-based models, it appears that it does not, in fact, provide better performance than our LSTM model, which, given that our set of data is relatively small and that our text size contains limited dependencies, means that rather than benefiting our model, it would likely deteriorate due to attention mechanism overperformance. These findings make it reasonable to assume that even in a small-scale classification of various human emotions, merely increasing model complexity does not necessarily provide a similar increase in performance. Furthermore, this difficulty in many models for sparse classification of minor human

emotions suggests that this problem not only applies to model complexity, but also that there are issues with imbalance and even with the notion of these emotions in general.

## 7 Limitations

Although our model performs reasonably well on the emotion classification task, it still has some limitations. First, RNN models are not very good at handling long range information. When a sentence is long, for example with more than 200 words, the model can forget the word came earlier, which hurts its accuracy and performance. Second, the dataset contains a lot of internet slang and abbreviations. Because the pre-trained word embeddings does not see these kinds of words before, the model often struggles with them. Another issue is that the emotion classes are not well balanced, some emotions appear much less than others, so the model tends to predict the more common categories. Finally, the model has trouble with sarcasm and irony, since these emotions are not expressed directly and usually require extra context or background knowledge to understand.

In the future, the model could be improved by using sub-word or character-based representations to reduce out-of-vocabulary (OOV) problems, trying resampling methods or different loss functions to fix class imbalance, and including emojis or other contextual information to better capture subtle emotions. We could also use a pre-trained model and use the idea of transfer learning, such as initializing our network by GloVe embedding.

## 8 Ethical considerations

The development and potential deployment of text-based emotion detection systems raises several ethical concerns that must be addressed carefully. First, the model relies on a large and typically imbalanced dataset, which can lead to systematic bias in model predictions, misclassifying emotional expressions in minority groups. Mitigating this issue requires strategic data augmentation techniques. Second, data privacy is a major concern when using data from online platforms, as there can be user data being used for research without consent. To mitigate this issue, our research relies solely on public and anonymized datasets, avoiding the collection of any personal information. Furthermore, these systems analyze text to infer a person's emotional state which is also a major privacy concern. Such systems can be misused for psychological profiling to exploit emotional vulnerabilities. For instance, organizations can use these predictions out of context and target individuals with specific content, crossing ethical boundaries by exploiting a person's emotional state without consent. To mitigate these risks, these systems should be used with full transparency with individuals who understand and agree with how their personal data is analyzed.

## 9 Conclusion

In this report, we investigated the efficacy of attention-based vs sequence-based architectures for fine grained emotion identification. In order to ascertain which architecture best conveys emotional nuance in brief, informal writing, we implemented and compared a Vanilla RNN, an LSTM RNN, and an Encoder-only Transformer.

With a Weighted F1-score of 0.898, our findings show that the LSTM model was the best architecture for this assignment. It regularly performed better than both the more complicated Transformer (86.85%) and the simpler Vanilla RNN (87.4%). Although Transformers are frequently regarded as the state of the art in natural language processing, our results indicate that the inductive bias of recurrent models, especially the LSTM model, provides better data efficiency and generalization for smaller datasets with short sequence lengths, especially for minority emotion classes like "surprise."

Our best model, however, still has problems with the dataset's intrinsic class imbalance and the semantic ambiguity of sarcasm and internet slang. Future research should concentrate on resolving these problems by utilizing resampling strategies to reduce class bias and including character based representations or sub-word tokenization, to handle out-of-vocabulary phrases, as mentioned in our limitations. Furthermore, the data limitations that hindered our Transformer's performance may be solved by utilizing transfer learning through huge language models or pre-trained embeddings (like GloVe), opening the door to a more reliable and sympathetic emotion detection system.

## References


[1] Govindaraj, P. (2020, April 16). Emotions dataset for NLP. Kaggle. https://www.kaggle.com/datasets/praveengovi/emotions-dataset-for-nlp

[2] Grekow, J. Music emotion recognition using recurrent neural networks and pretrained models. J Intell Inf Syst 57, 531–546 (2021). https://doi.org/10.1007/s10844-021-00658-5

[3] Bharti SK, Varadhaganapathy S, Gupta RK, Shukla PK, Bouye M, Hingaa SK, Mahmoud A. Text-Based Emotion Recognition Using Deep Learning Approach. Comput Intell Neurosci. 2022 Aug 23;2022:2645381. doi: 10.1155/2022/2645381. PMID: 36052029; PMCID: PMC9427219. https://doi.org/10.1155/2022/2645381